

```
In [ ]: def suggest_recipe():
    available_ingredients = []
    print("Please enter the available ingredients one by one. Type 'done' when finished.")
    while True:
        ingredient = input("Enter an ingredient: ").strip().lower()
        if ingredient == 'done':
            break
        available_ingredients.append(ingredient)

    recipes = {
        "Pasta Primavera": {
            "ingredients": ["pasta", "assorted vegetables", "olive oil", "garlic", "parmesan cheese"],
            "instructions": "\n1. Cook pasta according to package instructions. \n2. In a pan, sauté assorted vegetables and minced garlic in olive oil until tender. \n3. Toss cooked pasta with the vegetable mixture. \n4. Serve hot, garnish with parmesan cheese."
        },
        "Vegetable Stir-fry": {
            "ingredients": ["assorted vegetables", "soy sauce", "sesame oil", "garlic", "ginger", "rice"],
            "instructions": "\n1. Cook rice according to package instructions. \n2. In a wok or skillet, stir-fry assorted vegetables, minced garlic, and ginger in sesame oil until tender. \n3. Add cooked rice and soy sauce, stir-fry until well combined. \n4. Drizzle with soy sauce and sesame oil."
        },
        "Caprese Salad": {
            "ingredients": ["tomatoes", "fresh mozzarella", "fresh basil leaves", "balsamic vinegar", "olive oil"],
            "instructions": "\n1. Slice tomatoes and fresh mozzarella cheese into rounds. \n2. Arrange alternating slices of tomatoes, mozzarella, and basil leaves on a serving plate. \n3. Drizzle with balsamic vinegar and olive oil. \n4. Serve chilled."
        },
        "Tomato Basil Pasta": {
            "ingredients": ["pasta", "tomatoes", "garlic", "fresh basil leaves", "olive oil"],
            "instructions": "\n1. Cook pasta according to package instructions. \n2. In a pan, sauté chopped tomatoes and minced garlic in olive oil until tomatoes break down. \n3. Toss cooked pasta with the tomato mixture and torn fresh basil leaves. \n4. Drizzle with olive oil."
        },
        "Egg Fried Rice": {
            "ingredients": ["rice", "eggs", "assorted vegetables", "soy sauce", "sesame oil", "garlic"],
            "instructions": "\n1. Cook rice according to package instructions and let it cool. \n2. In a wok or skillet, scramble eggs in sesame oil. \n3. Add minced garlic and stir-fry assorted vegetables until tender. \n4. Add cooked rice and soy sauce, stir-fry until well combined. \n5. Drizzle with soy sauce and sesame oil."
        },
        "Chana Masala": {
            "ingredients": ["chickpeas", "onion", "tomatoes", "ginger", "garlic", "coriander", "cumin", "garam masala", "turmeric"],
            "instructions": "\n1. Heat oil in a pan and add chopped onions. Saute until golden brown. \n2. Add ginger and garlic paste, sauté until the raw smell disappears. \n3. Add chopped tomatoes and cook until they turn mushy. \n4. Add chickpeas, spices, and water. Simmer for 15-20 minutes. \n5. Garnish with coriander leaves."
        },
        "Paneer Tikka Masala": {
            "ingredients": ["paneer", "onion", "bell peppers", "yogurt", "tomato puree", "ginger-garlic paste", "kasuri methi", "garam masala", "turmeric", "coriander powder", "cumin powder"],
            "instructions": "\n1. Marinate paneer cubes in yogurt, ginger-garlic paste, turmeric, and salt. Let it sit for 30 minutes. \n2. Grill or roast marinated paneer cubes until golden brown. \n3. In a pan, heat oil and add chopped onions, bell peppers, and ginger. \n4. Add tomato puree, spices, and water. Simmer for 10-15 minutes. \n5. Add paneer cubes and kasuri methi. \n6. Garnish with coriander leaves."
        },
        "Chicken Curry": {
            "ingredients": ["chicken", "onion", "tomatoes", "ginger", "garlic", "chilli powder", "turmeric", "coriander powder", "cumin powder", "garam masala", "coconut milk"],
            "instructions": "\n1. Heat oil in a pan and add chopped onions. Saute until golden brown. \n2. Add ginger and garlic paste, sauté until the raw smell disappears. \n3. Add chopped tomatoes and cook until they turn mushy. \n4. Add chicken pieces, spices, and water. Simmer for 20-25 minutes. \n5. Add coconut milk and coriander leaves."
        },
        "Palak Paneer": {
            "ingredients": ["paneer", "spinach", "onion", "tomato", "ginger", "garlic", "green chilies", "garam masala", "cumin", "coriander powder", "turmeric"],
            "instructions": "\n1. Blanch spinach leaves in boiling water for 2-3 minutes. Drain and blend into a smooth puree. \n2. Heat oil in a pan, add cumin seeds, chopped onions, ginger, garlic, and green chilies. Saute until onions turn golden brown. \n3. Add tomato puree, spices, and water. Simmer for 10-15 minutes. \n4. Add paneer cubes and spinach puree. \n5. Garnish with coriander leaves."
        },
        "Dal Tadka": {
            "ingredients": ["yellow lentils", "onion", "tomato", "ginger", "garlic", "green chilies", "turmeric", "cumin seeds", "mustard seeds", "asafoetida", "garam masala"],
            "instructions": "\n1. Rinse lentils and cook with water, turmeric, salt, and chopped tomatoes until soft. Mash slightly. \n2. Heat ghee in a pan, add cumin seeds, mustard seeds, asafoetida, chopped onions, ginger, garlic, and green chilies. Saute until onions turn golden brown. \n3. Add the dal and garam masala. \n4. Garnish with coriander leaves."
        },
        "Aloo Gobi": {
            "ingredients": ["potatoes", "cauliflower", "onion", "tomato", "ginger", "garlic", "green chilies", "turmeric", "coriander powder", "cumin seeds", "garam masala"],
            "instructions": "\n1. Heat oil in a pan, add cumin seeds, chopped onions, ginger, garlic, and green chilies. Saute until onions turn golden brown. \n2. Add chopped tomatoes and cook until they soften. \n3. Add turmeric, coriander powder, and water. Simmer for 10-15 minutes. \n4. Add potato and cauliflower cubes. \n5. Garnish with coriander leaves."
        },
        "Vegetable Biryani": {
            "ingredients": ["rice", "assorted vegetables", "onion", "tomato", "ginger", "garlic", "green chilies", "yogurt", "mint leaves", "coriander leaves", "garam masala", "turmeric", "saffron"],
            "instructions": "\n1. Soak rice in water for 30 minutes. Drain and set aside. \n2. Heat oil in a pan, add cumin seeds, chopped onions, ginger, garlic, and green chilies. Saute until onions turn golden brown. \n3. Add assorted vegetables, tomato puree, spices, and water. Simmer for 10-15 minutes. \n4. Add saffron and yogurt. \n5. Layer with rice and vegetables. \n6. Cook on low heat until rice is done. \n7. Garnish with coriander leaves and mint leaves."
        },
        "Chicken Biryani": {
            "ingredients": ["rice", "chicken", "onion", "tomato", "ginger", "garlic", "green chilies", "yogurt", "mint leaves", "coriander leaves", "garam masala", "turmeric", "saffron"],
            "instructions": "\n1. Soak rice in water for 30 minutes. Drain and set aside. \n2. Marinate chicken pieces in yogurt, ginger-garlic paste, green chilies, turmeric, and garam masala. Let it sit for 30 minutes. \n3. Heat oil in a pan, add cumin seeds, chopped onions, ginger, garlic, and green chilies. Saute until onions turn golden brown. \n4. Add tomato puree, spices, and water. Simmer for 10-15 minutes. \n5. Add marinated chicken. \n6. Layer with rice. \n7. Cook on low heat until rice is done. \n8. Garnish with coriander leaves and mint leaves."
        },
        "Pani Puri": {
            "ingredients": ["semolina flour", "all-purpose flour", "potatoes", "chickpeas", "tamarind pulp", "mint leaves", "green chilies", "chaat masala", "cumin powder", "coriander powder", "black salt"],
            "instructions": "\n1. Prepare a dough using semolina flour, all-purpose flour, and water. Let it rest for 30 minutes. \n2. Roll out small puris from the dough and deep fry until they puff up. \n3. Boil and mash potatoes, and prepare a chutney using chickpeas, tamarind pulp, and spices. \n4. Fill the puris with potato mash, chutney, and chaat masala. \n5. Garnish with green chilies and coriander leaves."
        },
        "Masoor Dal": {
            "ingredients": ["red lentils", "onion", "tomato", "ginger", "garlic", "green chilies", "turmeric", "cumin seeds", "mustard seeds", "curry leaves", "asafoetida", "garam masala"],
            "instructions": "\n1. Rinse lentils and cook with water, turmeric, salt, and chopped tomatoes until soft. Mash slightly. \n2. Heat oil in a pan, add cumin seeds, mustard seeds, asafoetida, chopped onions, ginger, garlic, green chilies, and curry leaves. Saute until onions turn golden brown. \n3. Add the dal and garam masala. \n4. Garnish with coriander leaves."
        },
        "Vegetable Pakora": {
            "ingredients": ["gram flour", "assorted vegetables", "onion", "ginger", "garlic", "green chilies", "coriander leaves", "cumin seeds", "carom seeds", "turmeric", "chili powder", "garam masala"],
            "instructions": "\n1. Mix gram flour, chopped onions, ginger, garlic, green chilies, coriander leaves, cumin seeds, carom seeds, turmeric, chili powder, garam masala, and salt in a bowl. Add water to make a thick batter. \n2. Heat oil in a deep fryer. \n3. Dip vegetables in the batter and fry until golden brown. \n4. Drain on a paper towel. \n5. Serve hot with a dipping sauce."
        }
    }

    threshold = 0.6 # 60%

    suggested_recipes = []
    for recipe, details in recipes.items():
        required_ingredients = details["ingredients"]
        matching_ingredients = sum(1 for ingredient in available_ingredients if ingredient in required_ingredients)
        if matching_ingredients / len(required_ingredients) >= threshold:
            suggested_recipes.append((recipe, details["instructions"]))

    if suggested_recipes:
        print("\nBased on your available ingredients, you can try making:")
        for recipe, instructions in suggested_recipes:
            print(f"{recipe}")
            print(f"{instructions}")
        else:
            print("\nSorry, with the available ingredients, I couldn't find a suitable recipe.")

    # Call the function to run the program
    suggest_recipe()
```

```
In [4]: import pandas as pd

In [5]: data = pd.read_csv("Untitled form.csv")

In [6]: data.head()
```

Out[6]:

	Timestamp	NAME	AGE	GENDER	TIME OF THE DAY	MOOD	CRAVINGS	ALLERGIES IF ANY:
0	2025/04/29 7:22:14 pm GMT+5:30	Aishwarya	20-30	FEMALE	MORNING	NEUTRAL	Pancakes	Soya
1	2025/04/29 7:22:42 pm GMT+5:30	Ritesh	10-20	MALE	AFTERNOON	POSITIVE	Biryani	Nil
2	2025/04/29 7:23:22 pm GMT+5:30	Sathish	20-30	MALE	EVENING	POSITIVE	Pizza	Nuts
3	2025/04/29 7:24:33 pm GMT+5:30	Sharav	30-40	MALE	EVENING	NEGATIVE	Ice cream	Prawns
4	2025/04/29 7:25:02 pm GMT+5:30	Mona shree	10-20	FEMALE	AFTERNOON	POSITIVE	Biryani	Peanuts

```
In [7]: data = data.drop(columns=["Timestamp", "NAME"])

In [8]: data.head()
```

Out[8]:

	AGE	GENDER	TIME OF THE DAY	MOOD	CRAVINGS	ALLERGIES IF ANY:
0	20-30	FEMALE	MORNING	NEUTRAL	Pancakes	Soya
1	10-20	MALE	AFTERNOON	POSITIVE	Biryani	Nil
2	20-30	MALE	EVENING	POSITIVE	Pizza	Nuts
3	30-40	MALE	EVENING	NEGATIVE	Ice cream	Prawns
4	10-20	FEMALE	AFTERNOON	POSITIVE	Biryani	Peanuts

```
In [9]: # Map age ranges to numerical values
age_mapping = {
    '<10': 0,
    '10-20': 1,
    '20-30': 2,
    '30-40': 3,
    '40-50': 4,
    '50-60': 5,
    '>60': 6
}
data['AGE'] = data['AGE'].map(age_mapping)

In [10]: data.shape

Out[10]: (40, 6)

In [11]: data.columns

Out[11]: Index(['AGE', 'GENDER', 'TIME OF THE DAY', 'MOOD', 'CRAVINGS', 'ALLERGIES IF ANY:'],
              dtype='object')
```

```
In [12]: from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, accuracy_score
```

```
In [13]: # Encode categorical features
label_encoder = LabelEncoder()
data['GENDER'] = label_encoder.fit_transform(data['GENDER'])
data['TIME OF THE DAY'] = label_encoder.fit_transform(data['TIME OF THE DAY'])
data['MOOD'] = label_encoder.fit_transform(data['MOOD'])
data['ALLERGIES IF ANY:'] = label_encoder.fit_transform(data['ALLERGIES IF ANY:'])
# Handle missing values if any
data.fillna(0, inplace=True) # Using 0 as a simple fill strategy
```

```
In [14]: # 2. Model Training
X = data[['AGE', 'GENDER', 'TIME OF THE DAY', 'MOOD']]
y = data['CRAVINGS']

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
In [15]: # Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
In [16]: # Initialize and train the KNN model
k = 1
knn_model = KNeighborsClassifier(n_neighbors=k)
knn_model.fit(X_train_scaled, y_train)
```

```
Out[16]: KNeighborsClassifier(n_neighbors=1)
```

```
In [17]: y_pred = knn_model.predict(X_test_scaled)
```

```
In [18]: # 3. User Input and Prediction
print("--- Craving Prediction ---")
try:
    # The script will wait here for your first input
    user_age = int(input("Enter age: "))

    if user_age < 10: user_age_range = '<10'
    elif user_age < 20: user_age_range = '10-20'
    elif user_age < 30: user_age_range = '20-30'
    elif user_age < 40: user_age_range = '30-40'
    elif user_age < 50: user_age_range = '40-50'
    elif user_age < 60: user_age_range = '50-60'
    else: user_age_range = '>60'

    user_age_encoded = age_mapping[user_age_range]

    # Then it will wait here for your second input
    user_gender = input("Enter gender (MALE/FEMALE/OTHERS): ").upper()
    # And finally, it will wait here for your third input
    user_time_of_day = input("Enter time of the day (MORNING/AFTERNOON/EVENING/NIGHT): ").upper()
    user_mood = input("Enter mood (POSITIVE/NEGATIVE/NEUTRAL): ").upper()

    # Mappings for user inputs
    gender_mapping = {'MALE': 0, 'FEMALE': 1, 'OTHERS': 2}
    time_of_day_mapping = {'MORNING': 0, 'AFTERNOON': 1, 'EVENING': 2, 'NIGHT': 3}
    mood_mapping = {'POSITIVE': 0, 'NEGATIVE': 1, 'NEUTRAL': 2}

    user_gender_encoded = gender_mapping.get(user_gender, -1)
    user_time_of_day_encoded = time_of_day_mapping.get(user_time_of_day, -1)
    user_mood_encoded = mood_mapping.get(user_mood, -1)

    if -1 in [user_gender_encoded, user_time_of_day_encoded, user_mood_encoded]:
        print("Invalid Input. Please check your spelling and try again.")
    else:
        user_input = [[user_age_encoded, user_gender_encoded, user_time_of_day_encoded, user_mood_encoded]]
        user_input_scaled = scaler.transform(user_input)
        predicted_craving = knn_model.predict(user_input_scaled)
        print(f"Predicted craving: {predicted_craving[0]}")

except ValueError:
    print("Invalid age entered. Please enter a whole number.")
except Exception as e:
    print(f"An unexpected error occurred: {e}")
```

```
--- Craving Prediction ---
Predicted craving: Coffee
C:\Users\rishanaconda3\Lib\site-packages\sklearn\utils\validation.py:2739: UserWarning: X does not have valid feature names, but StandardScaler was fitted with feature names
warnings.warn(
```

```
In [19]: import joblib
# knn_model = KNeighborsClassifier(n_neighbors=k)
# knn_model.fit(X_train_scaled, y_train)
# ADD THESE TWO LINES TO SAVE YOUR FILES:
print("Saving model...")
joblib.dump(knn_model, 'knn_model.pkl')
joblib.dump(scaler, 'scaler.pkl')
print("Model and scaler saved!")
```

```
Saving model...
Model and scaler saved!
```

```
In [ ]:
```